FACULTY OF ENGINEERING & TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE
COMP333
DATA BASE SYSTEM

Project Report

Prepared By: Israa Turkman 1221003
Donia Said 1222600

Section: 5

Date: 18-01-2026

Project Title:

I&D Lab– Laboratory Equipment Sales Management System

Project Brief Description

I&D Lab is a database-driven web application designed to manage the sales, inventory, and purchasing operations of a company that sells medical and chemical laboratory equipment and supplies. The system supports both customer and administrative operations in order to keep data consistent and improve the efficiency of daily work.

Customers can register, log in, browse available products, place sales orders, and review their order history. Administrators can manage the product catalog and monitor business performance through reports and statistics generated directly from the database.

Main Functionality

The system provides the following main functions:

Customer Functions

- Customer registration and login.
- Browsing laboratory products and viewing product details.
- Adding products to a shopping cart and placing a sales order.
- Viewing order history and order details.

Administrative Functions

- Product management (Add / Edit / Delete / List products).
- Viewing database-driven reports and statistics (KPIs, charts, and summary tables).
- Managing the core master data used in the system (branches/warehouses/suppliers as part of the database design).

Entity-Relationship (ER) Diagram

The database design of the I&D Lab system is based on a well-structured Entity-Relationship (ER) model that represents all core business entities and their relationships. The ER diagram was designed to support sales, inventory management, purchasing, and reporting operations while maintaining data integrity and minimizing redundancy.

The diagram clearly defines primary keys, foreign keys, weak entities, and relationship cardinalities.

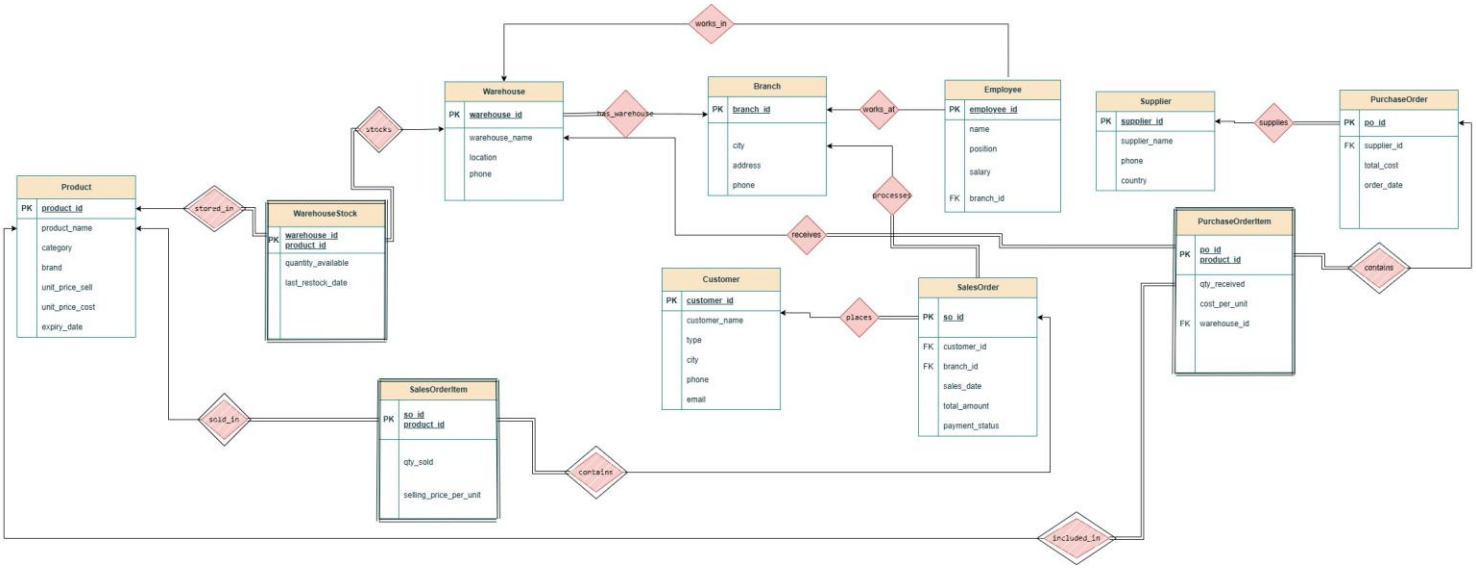

Figure 1: ER Daigram.

Entity Sets

The system includes the following entity sets:

- **Customer**: Stores customer information such as name, type, city, phone number, and email.
- **Product**: Represents laboratory products including category, brand, selling price, cost price, and expiry date.
- **SalesOrder**: Stores customer sales orders with order date, total amount, and payment status.
- **SalesOrderItem**: A weak entity that represents products included in a sales order along with quantity and selling price.
- **Branch**: Represents company branches with city, address, and contact information.
- **Warehouse**: Stores warehouse details such as name, location, and phone number.
- **WarehouseStock**: A weak entity that tracks product quantities available in each warehouse.
- **Supplier**: Stores supplier information including name, phone number, and country.
- **PurchaseOrder**: Represents purchase orders made to suppliers.
- **PurchaseOrderItem**: A weak entity that records products received through purchase orders.
- **Employee**: Represents employees working in branches.

Relationship Sets

The following relationships exist between the entity sets:

- Product stored_in WarehouseStock (1:N)
- Warehouse stocks WarehouseStock (1:N)
- Branch has_warehouse Warehouse (1:N)
- Employee works_at Branch (N:1)
- Employee works_in Warehouse (N:1)
- Branch receives PurchaseOrderItem (1:N)
- Customer places SalesOrder (1:N)
- Branch processes SalesOrder (1:N)
- SalesOrder contains SalesOrderItem (1:N)
- Product sold_in SalesOrderItem (1:N)
- Supplier supplies PurchaseOrder (1:N)
- PurchaseOrder contains PurchaseOrderItem (1:N)
- PurchaseOrderItem included_in Product (N:1)

Weak Entities

The system includes the following weak entities:

- **SalesOrderItem**: Depends on *SalesOrder* and *Product* and uses a composite primary key (so_id, product_id).
- **WarehouseStock**: Depends on *Warehouse* and *Product* and uses a composite primary key (warehouse_id, product_id).
- **PurchaseOrderItem**: Depends on *PurchaseOrder* and *Product* and uses a composite primary key (po_id, product_id).

These weak entities cannot exist independently and rely on their owning entities to maintain referential integrity.

Relational Schema

- **Branch**

  Branch (**branch_id**, city, address, phone)
- **Warehouse**

  Warehouse (**warehouse_id**, warehouse_name, location, phone, branch_id)
- **Employee**

  Employee (**employee_id**, name, position, salary, branch_id)
- **Customer**

  Customer (**customer_id**, customer_name, type, city, phone, email, password_hash)
- **Product**

  Product (**product_id**, product_name, category, brand, unit_price_sell, unit_price_cost, expiry_date)
- **WarehouseStock**

  WarehouseStock (**warehouse_id**, **product_id**, quantity_available, last_restock_date)
- **SalesOrder**

  SalesOrder (**so_id**, customer_id, branch_id, sales_date, total_amount, payment_status)
- **SalesOrderItem**

  SalesOrderItem (**so_id**, **product_id**, qty_sold, selling_price_per_unit)
- **Supplier**

  Supplier (**supplier_id**, supplier_name, phone, country)
- **PurchaseOrder**

  PurchaseOrder (**po_id**, supplier_id, total_cost, order_date)
- **PurchaseOrderItem**

  PurchaseOrderItem (**po_id**, **product_id**, qty_received, cost_per_unit, warehouse_id)

Normalization

1. First Normal Form (1NF)

Requirements:

- Each column must contain atomic values.
- There should be no repeating groups.
- Each table must have a primary key.

Analysis:

All tables in the system satisfy 1NF because:

- All attributes are atomic (INT, VARCHAR, DATE, DECIMAL).
- No table contains multi-valued attributes.
- Each table has a defined primary key.

Conclusion:

All tables are in 1NF.

2. Second Normal Form (2NF)

Second Normal Form applies mainly to tables with composite primary keys.

Table: WarehouseStock

Primary Key: (warehouse_id, product_id)

Other Attributes: quantity_available, last_restock_date

Analysis:

Both quantity_available and last_restock_date depend on the combination of warehouse_id and product_id, since stock quantity is specific to a product in a specific warehouse.

There is no partial dependency.

Conclusion: In **2NF**

Table: SalesOrderItem

Primary Key: (so_id, product_id)

Other Attributes: qty_sold, selling_price_per_unit

Analysis:

The quantity sold and selling price depend on both the sales order and the product together.

No attribute depends on only part of the primary key.

Conclusion: In **2NF**

Table: PurchaseOrderItem

Primary Key: (po_id, product_id)

Other Attributes: qty_received, cost_per_unit, warehouse_id

Analysis:

All non-key attributes depend on the combination of purchase order and product.

There are no partial dependencies.

Conclusion: In **2NF**

All Other Tables:
Tables such as Customer, Product, Branch, Warehouse, Supplier, Employee, SalesOrder, and PurchaseOrder have single-column primary keys, so partial dependency does not exist.

3. Third Normal Form (3NF)

Third Normal Form requires that no non-key attribute depends on another non-key attribute.

Table: Customer

Primary Key: customer_id

Attributes: customer_name, type, city, phone, email

Analysis:

All attributes depend directly on customer_id.

There are no transitive dependencies.

Conclusion: In **3NF**

Table: Product

Primary Key: product_id

Attributes: product_name, category, brand, unit_price_sell, unit_price_cost, expiry_date

Analysis:

All attributes describe the product identified by product_id.

No non-key attribute depends on another non-key attribute.

Conclusion: In **3NF**

Table: SalesOrder

Primary Key: so_id

Attributes: customer_id, branch_id, sales_date, total_amount, payment_status

Analysis:

All attributes depend directly on so_id.

Foreign keys are used correctly and do not create transitive dependencies.

Conclusion: In **3NF**

Table: Branch

Primary Key: branch_id

Attributes: city, address, phone

Analysis:

All attributes describe the branch identified by branch_id.

There are no transitive dependencies.

Conclusion: In **3NF**

Table: Warehouse

Primary Key: warehouse_id

Attributes: warehouse_name, location, phone

Conclusion: In **3NF**

Table: Supplier

Primary Key: supplier_id

Attributes: supplier_name, phone, country

Conclusion: In **3NF**

Table: Employee

Primary Key: employee_id

Attributes: name, position, salary, branch_id

Analysis:

All attributes depend on employee_id.

branch_id is a foreign key and does not cause transitive dependency.

Conclusion: In **3NF**

Table: PurchaseOrder

Primary Key: po_id

Attributes: supplier_id, total_cost, order_date

Conclusion: In **3NF**

Table: WarehouseStock

Primary Key: (warehouse_id, product_id)

Attributes: quantity_available, last_restock_date

Analysis:

All attributes depend directly on the composite primary key (warehouse_id, product_id). There are no non-key attributes that depend on other non-key attributes, so there are no transitive dependencies.

Conclusion: In **3NF**

Table: SalesOrderItem

Primary Key: (so_id, product_id)

Attributes: qty_sold, selling_price_per_unit

Analysis:

Both qty_sold and selling_price_per_unit depend directly on the composite key (so_id, product_id).

There are no transitive dependencies.

Conclusion: In **3NF**

Table: PurchaseOrderItem

Primary Key: (po_id, product_id)

Attributes: qty_received, cost_per_unit, warehouse_id

Analysis:

All attributes depend directly on the composite primary key (po_id, product_id).

The attribute warehouse_id is a foreign key used for reference only and does not create a transitive dependency.

Conclusion: In **3NF**

Queries

A. Sales & Customer Queries

1. **Retrieve** customer orders within a specified date range for a specific branch city, including order ID, customer name, branch city, order date, total amount, and payment status.
2. **Retrieve** a sales summary grouped by product category within a specified date range, showing total units sold and total revenue per category.
3. **Retrieve** the order history of a specific customer, including order date, payment status, number of items in each order, and the computed total amount per order.
4. **Retrieve** detailed information for a specific sales order, including product name, category, quantity sold, unit selling price, and line total.
5. **Retrieve** total spending grouped by customer type (such as University, Hospital, etc.), including total number of orders and total amount spent.
6. **Retrieve** best-selling products per branch, including total quantity sold and total revenue for each product in each branch.

B. Inventory & Purchase Queries

7. **Retrieve** low-stock products in warehouses for a specific branch city, showing warehouse name, product name, and available quantity below a defined threshold.
8. **Retrieve** purchase orders from a specific supplier within a given date range and received by a specific warehouse, including product details, quantities received, unit cost, and total line cost.
9. **Retrieve** total purchases per supplier, including number of purchase orders and total purchased amount.
10. **Retrieve** the most purchased products based on total received quantities, grouped by product and category.
11. **Retrieve** inventory value per branch by summing the value of all products stored in warehouses under each branch.
12. **Retrieve** warehouse stock summaries, showing the number of different products stored in each warehouse and the total available quantity.

CRUD Queries (Insert, Update, Delete)

C. Customer Management Queries

13. **Insert** a new customer account with personal details such as name, city, phone number, email, and encrypted password.
14. **Retrieve** customer information using email during the login process for authentication purposes.

D. Product Management Queries

15. **Insert** a new product into the system with details such as product name, category, brand, selling price, cost price, and expiration date.
16. **Update** existing product information including name, category, brand, prices, and expiration date.
17. **Delete** a product from the system based on its unique product ID.

E. Branch Management Queries

18. **Retrieve** all branch details including city, address, and phone number for administrative management.
19. **Update** branch information such as city, address, and phone number.
20. **Delete** a branch from the system using the branch ID.

F. Supplier Management Queries

21. **Retrieve** the list of all suppliers with their contact details.
22. **Insert** a new supplier with name, phone number, and country.
23. **Update** supplier information such as name, phone number, and country.
24. **Delete** a supplier from the system.

G. Warehouse Management Queries

25. **Retrieve** warehouse information along with the branch it belongs to.
26. **Insert** a new warehouse and associate it with a specific branch.
27. **Update** warehouse details such as name, location, phone number, and branch assignment.
28. **Delete** a warehouse from the system.

H. Stock Management Queries

29. **Retrieve** current stock levels for products in a specific warehouse.
30. **Update** product stock quantities during restocking operations and update the last restock date.
31. **Retrieve** products that are low in stock across all warehouses.

I. Purchase Order Management Queries

32. **Insert** a new purchase order for a selected supplier.

33. **Insert** purchase order items including product, warehouse, received quantity, and unit cost.
34. **Update** warehouse stock quantities automatically after receiving purchased items.
35. **Update** the total cost of a purchase order based on its items.
36. **Delete** a purchase order from the system.

J. Sales Order Management Queries

37. **Insert** a new sales order when a customer completes checkout.
38. **Insert** sales order items including product, quantity sold, and selling price per unit.
39. **Retrieve** all orders placed by a specific customer.
40. **Retrieve** detailed items of a specific sales order.